Il linguaggio SQL (Structured Query Language). Fondamenti e Pattern d'uso

SQL: definizione

- SQL (*Structured Query Language*) è un linguaggio specializzato per la gestione di basi di dati **relazionali**
- È utilizzato per la creazione e la modifica di schemi, nonché per la gestione e l'interrogazione dei dati
- Standardizzato e ampiamente utilizzato nell'ambito della gestione dei dati

SQL: DDL e DML

- DDL (Data Definition Language): Utilizzato per definire la struttura della base di dati
  - Creazione di tabelle, modifiche dello schema, definizione di vincoli.

- DML (Data Manipulation Language): Utilizzato per manipolare i dati all'interno del database.
  - Inserimento, aggiornamento, cancellazione e interrogazione dei dati.

SQL: molto più di un *query language*

- Per molti, SQL è un modo per estrarre tuple da un database relazionale
- Ma chi usa SQL per lavoro sa che è più un modello cognitivo per interagire con sistemi complessi e accedere a informazione complessa
- E' certamente una sintassi, ma il fine ultimo è quello di trasformare le richieste della *business logic* in informazione che risponde a domande di interesse per l'azienda

Per esempio ….

- Domanda (*business logic*): quanti sono stati gli utenti attivi del mese di agosto 2025?
- Query SQL:

```
SELECT count(user_id)
FROM events
WHERE event_date between '2025-08-01’ and '2025-08-31’
AND event_type = 'login’;
```

- Questa query funziona, ma ignora questioni come:
  - Come consideriamo gli utenti mobile che consumano dati senza loggarsi ogni volta?
  - Vogliamo considerare o no gli utenti di test?
  - I dati sono sensibili al fuso orario?

SQL e IA

- Con i vari ChatGPT, Claude, Gemini, …, scrivere query è diventata la parte più semplice del lavoro di un *data analyst* (vedi Text-2-SQL)
- Il suo ruolo quindi (quello per cui è pagato) non può essere (solo) quello di conoscere perfettamente tutti i comandi SQL, ma di:
  - Riconoscere assunzioni profonde nelle interrogazioni
  - Raffinare le domande, in modo da rendere le risposte più incisive
  - Tradurre un problema di business in una logica che permetta di (estrarre i dati giusti per) risolverlo

Ancora un esempio …

Domanda (*business logic*): *Se stessi analizzando l'abbandono dei clienti per un'attività in abbonamento, come lo affronteresti*?

Il problema non è la sintassi della query, ma comprendere quali sono le domande che fornirebbero una risposta al problema e poi convertirle in operatori SQL

Ancora un esempio …

- **Definire cosa si intende per “abbandono”**
  - Nessun accesso negli ultimi 30 giorni?
  - Nessun acquisto?
  - Cancellazione dell’account?
- **Spezzare il problema in sotto-problemi:**
  - Trovare l’ultima attività di ogni cliente (MAX sulla data)
  - Confrontare con la data corrente (DATEDIFF)
  - Filtrare i clienti dove la differenza > 30
- **Solo allora possiamo iniziare a mappare le domande sugli operatori SQL, per esempio:**
  - FROM → la tabella delle attività dei clienti
  - GROUP BY → per raggruppare per utente (via customer_id)
  - HAVING → per la soglia temporale di non attività

Riferimenti

- Per una presentazione sistematica dei comandi di SQL, potete fare riferimento ai seguenti siti:
  - Manuale introduttivo (interattivo) SQL: https://www.w3schools.com/mysql/default.asp
  - MariaDB Server Documentation: https://mariadb.com/docs/server
  - *Reference manual* ufficiale di MySQL: https://dev.mysql.com/doc/refman/8.4/en/tutorial.html
- Nelle lezioni discuteremo soprattutto di pattern e casi d'uso di SQL, quindi usate i manuali come referenza per la sintassi!

Strumenti per lavorare con RDBMS

- Installare sul proprio computer un DBMS a scelta tra:
  - MariaDB: https://mariadb.org/download/
  - MySQL: https://www.mysql.com/downloads/
  - [PostgreSQL: https://www.postgresql.org/download/]

- Installare un'interfaccia grafica per l'utilizzo del DBMS:
  - Dbeaver: https://dbeaver.io/download/ («a cross-platform database tool for developers, database administrators, analysts, and everyone working with data. It supports all popular SQL databases like MySQL, MariaDB, PostgreSQL, SQLite, Apache Family, and more»)
  - MySQL workbench: https://dev.mysql.com/downloads/workbench/
  - [pgAdmin: https://www.pgadmin.org/]

Schemi e dati per esercitarsi

- Sul sito del corso sono a disposizione gli script SQL per generare schema e dati di due database relazionali:
  - La base di dati «**Sailors**», che è lo schema utilizzato nel libro di testo e nelle slide per presentare SQL
  - La base di dati «**NorthWind**», che è uno schema pubblico con una buona quantità di dati nel settore degli ordini di un'azienda
- Per entrambi gli schemi verranno fornite liste di esercizi (query)

Istanza DB Sailors utilizzata nel seguito

sid	*sname*	*rating*	*age*
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 5.1 An Instance $S3$ of Sailors

sid	*bid*	*day*
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 5.2 An Instance $R2$ of Reserves

bid	*bname*	*color*
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Figure 5.3 An Instance $B1$ of Boats

STRUTTURA GENERALE DELLE QUERY SQL

Struttura base

SELECT	[DISTINCT] *target-list*
FROM	*relation-list*
WHERE	*qualification*

- *relation-list*: lista di nomi di relazioni – da 1 a *n*
- *target-list*: lista di attributi che appartengono a relazioni nella *relation-list*
- *qualification*: operatori di confronto ($<, >, =, \leq, \geq, \neq$) eventualmente combinati con AND, OR o NOT
- DISTINCT: chiave opzionale per indicare che i duplicati devono esser eliminati dal risultato dell'interrogazione (il *default* è che NON vengono eliminati → **diversamente da algebra relazionale**!!)

Strategia concettuale di valutazione

SELECT	[DISTINCT] *target-list*
FROM	*relation-list*
WHERE	*qualification*

- FROM: si calcola il prodotto cartesiano delle tabelle nella *relation-list*
- WHERE: si scartano le tuple che non soddisfano le *qualifications*
- SELECT: si cancellano le colonne degli attributi che non appartengono alla *target-list*
- DISTINCT: si eliminano i duplicati
- [ORDER BY: si ordinano le righe o i gruppi risultanti]

NB: questa è una strategia **molto inefficiente** per valutare la query!! L'ottimizzatore dei DBMS serve proprio a creare piani di esecuzione più efficienti (come vedremo in seguito)

Esempi

Alias per attributo in output

```
select * from sailors s;
```



	sid	sname	rating	age
1	22	Dustin	7	45
2	29	Brutus	1	33
3	31	lubber	8	55,5
4	32	Andy	8	25,5
5	58	Rusty	10	35
6	64	Horatio	7	35
7	71	Zorba	10	16
8	74	Horatio	9	35
9	85	Art	3	22,5
10	95	Bob	3	63,5

```
select * from sailors s
where rating = 7;
```



	sid	sname	rating	age
1	22	Dustin	7	45
2	64	Horatio	7	35

Esempi

Alias per attributi in output

```
select s.rating as Valutazione,
       s.sname as Nome
from sailors s
where s.sname = 'Horatio'
      or rating > 7;
```



	Valutazione	Nome
1	8	lubber
2	8	Andy
3	10	Rusty
4	7	Horatio
5	10	Zorba
6	9	Horatio

```
select * from sailors s
order by s.rating desc ;
```



	sid	sname	rating	age
1	58	Rusty	10	35
2	71	Zorba	10	16
3	74	Horatio	9	35
4	31	lubber	8	55,5
5	32	Andy	8	25,5
6	22	Dustin	7	45
7	64	Horatio	7	35
8	85	Art	3	22,5
9	95	Bob	3	63,5
10	29	Brutus	1	33

Operatori di confronto

- Le qualificazioni di una query SQL sono condizioni che si basano sull'uso di operatori di confronto
- In SQL i principali sono:
  - Operatori aritmetici: =, <, >, >=, <=, <>
  - **BETWEEN … AND …**: verifica se un valore è compreso in un intervallo
  - **IN**: verifica se un valore è compreso in una lista
  - **LIKE**: confronto fra stringhe con caratteri jolly (% e _)
  - **IS [NOT] NULL**: verifica se un valore ha (o non ha) valore NULL → […] **where x = NULL] è SBAGLIATO!!**

Esempi

```
select * from reserves r
where r.`day` between '1998-06-02' and '1998-08-31'
order by r.`day` ;
```

sid	bid	day
31	103	1998-06-11
22	104	1998-07-10
64	102	1998-08-09
74	103	1998-08-09
22	103	1998-08-10

```
select r.sid , r.`day`
from reserves r
where r.bid in(101, 103) ;
```

sid	day
22	1998-10-10
64	1998-05-09
22	1998-08-10
31	1998-06-11
74	1998-08-09

```
select s.sname
from sailors s
where s.sname like '%s_%' ;
```

sname
Dustin
Rusty

```
select * from boats b
where color is null ;
```

	bid	bname	color
1	105	Prova	[NULL]

Dall'estrazione all'analisi dei dati

- Molto spesso si pensa a SQL come a un linguaggio per estrarre tuple (o parte di tuple) da una o più relazioni (tabelle)
- Ma in realtà SQL è anche un potente strumento di analisi dei dati e di supporto alle decisioni *(decision support systems - DSS)*
- Gli operatori più importanti in tal senso sono gli **operatori di aggregazione**

Partiamo con un paio di esempi

Quali prenotazioni vs. quante prenotazioni?

- Alla prima domanda possiamo rispondere con la query

```
select * from reserves r ;
```

- Ma la seconda richiede di contare le prenotazioni, non di elencarle!

```
select count(*) from reserves r ;
```

«Quali sono le valutazioni dei marinai» *vs.* «Qual è la media delle valutazioni»

- Alla prima domanda possiamo rispondere con la query

```
select s.sname, s.rating
from sailors s;
```

- La seconda richiede di calcolare una media tra tutte le valutazioni, non semplicemente di elencarle!

```
select avg(s.rating)from reserves r ;
```

Operatori di aggregazione

- SQL supporta 5 operatori di aggregazione:
  - count(*): numero di tuple restituite da una query
  - count([DISTINCT] A): numero di valori [unici] di A
  - sum([DISTINCT] A): somma dei valori [unici] di A
  - avg([DISTINCT] A): media dei valori [unici] di A
  - MAX(A): il valore più alto nella colonna A
  - MIN(A): il valore più basso nella colonna A
- Questi operatori permettono di aggregare dati usando funzioni aritmetiche, andando oltre gli operatori dell'algebra relazionale

Esempi

```
select min(s.rating) as ValMinima,
       max(s.rating) as ValMassima,
       avg(s.rating) as ValMedia
from sailors s ;
```



	ValMinima	ValMassima	ValMedia
1	1	10	6,6

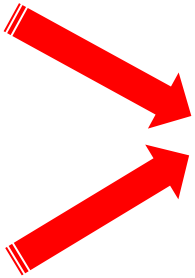
```
select count(*) as NumNoleggi
from reserves r
where r.bid = 103;
```



	NumNoleggi
1	3

```
select count(s.sname)
from sailors s ;

select count(distinct s.sname)
from sailors s ;
```



Che differenza c'è?

Struttura estesa: GROUP BY / HAVING

- Talvolta è necessario applicare operazioni di aggregazione non a tutte le tuple di una relazione, ma a gruppi di tuple raggruppate in base a un certo criterio
  - Esempio 1: contare quante volte è stata noleggiata ogni singola barca disponibile
  - Esempio 2: trovare l'età media dei marinai che hanno lo stesso rating nella tabella Sailors
- A questo scopo, la struttura di base delle query è estesa con le clausole GROUP BY / HAVING

GROUP BY e HAVING

```
SELECT    [ DISTINCT ] select-list
FROM      from-list
WHERE     qualification
GROUP BY  grouping-list
HAVING    group-qualification
```

- **select-list**: è composta da (1) una lista di nomi di colonne e (2) una lista di termini della forma **agg-op**(*nome-colonna*) AS *nuovo_nome* (il nuovo nome è utile, anche se non indispensabile)
- Ogni colonna che compare in (1) **deve** apparire anche nella **grouping-list** (onde evitare che nel gruppo quella colonna possa avere valori diversi)
- Le espressioni che compaiono nella **group-qualification** devono sempre avere un solo valore per gruppo (valore che viene usato per decidere se un gruppo deve essere restituito nel risultato)
- Se nella query non compare il GROUP BY, l'intera tabella viene considerata come un singolo gruppo

GROUP BY e HAVING: valutazione

```
SELECT    [ DISTINCT ] select-list
FROM      from-list
WHERE     qualification
GROUP BY  grouping-list
HAVING    group-qualification
```

- FROM / JOIN: si calcola il prodotto cartesiano delle tabelle nella *relation-list*
- WHERE: si scartano le tuple che non soddisfano le *qualifications*
- GROUP BY: partiziona le righe rimanenti in gruppi in base ai valori degli attributi specificati
- HAVING: filtra i gruppi creati applicando le *group qualifications*
- SELECT: si cancellano le colonne degli attributi che non appartengono alla *target-list*
- [ORDER BY: si ordinano le righe o i gruppi risultanti]

Esempi GROUP BY / HAVING

Per ogni rating, trovare l'età media dei marinai con quel rating

```
select s.rating, avg(s.age) as EtaMedia
from sailors s;
```



Errore!!

Perché questa query è sbagliata?

- Una clausola SELECT senza GROUP BY restituisce una risposta che deve appartenere a **uno e un solo livello di aggregazione**:
  - avg(s.age) calcola un **singolo valore numerico** aggregato per l'intera colonna *age* della tabella (es. l'età media globale di tutti i marinai, metti 35).
  - s.rating chiede di restituire i valori di una colonna che cambiano da riga a riga (es. 7,8,3,10, ...).
- Il motore SQL non sa come abbinare un **singolo numero sintetico** alle **righe distinte per ogni rating**.

Esempi GROUP BY / HAVING

Per ogni rating, trovare l'età media dei marinai con quel rating

```
select s.rating, avg(s.age) as EtaMedia
from sailors s
group by s.rating ;
```



	rating	EtaMedia
1	1	33
2	3	43
3	7	40
4	8	40,5
5	9	35
6	10	25,5

Numero di noleggi di ogni singola barca (se numero > 2)

```
select r.bid, count(*) as NumNoleggi
from reserves r
group by r.bid
having NumNoleggi > 2;
```



	bid	NumNoleggi
1	102	3
2	103	3

GROUP BY e HAVING: esempio

(Q32) Find the age of the youngest sailor who is eligible to vote (i.e., is at least 18 years old) for each rating level with at least two such sailors.

```
SELECT    S.rating, MIN (S.age) AS minage
FROM      Sailors S
WHERE     S.age >= 18
GROUP BY  S.rating
HAVING    COUNT (*) > 1
```

sid	*sname*	*rating*	*age*
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5
96	Frodo	3	25.5

Passi 1, 2 e 3

rating	*age*
7	45.0
1	33.0
8	55.5
8	25.5
10	35.0
7	35.0
9	35.0
3	25.5
3	63.5
3	25.5

Passo 4

rating	*age*
1	33.0
3	25.5
3	25.5
3	63.5
7	45.0
7	35.0
8	55.5
8	25.5
9	35.0
10	35.0

Passi 5 e 6

rating	*minage*
3	25.5
7	35.0
8	25.5

ANNIDAMENTO DI QUERY

Intuizione di base

- Supponiamo di voler trovare il nome del marinaio (o dei marinai) che ha (hanno) la valutazione più alta
- Il primo istinto potrebbe essere di scrivere una query del tipo:

```
select s.sname
from sailors s
where s.rating = max(s.rating );
```

- Funziona? Se no, perché?
- [Hint: come funziona il WHERE?]

Interrogazioni annidate (*nested queries*)

- Una query annidata è una query (detta *interna*) che compare in un'altra query (detta *esterna*)
- Per esempio:

```
select s.sname
from sailors s
where s.rating = (select max(s2.rating)
                  from sailors s2 );
```

- Semanticamente, l'idea è simile a quella di un *nested loop*: per ogni tupla di Sailor nella *query esterna*, viene valutata la condizione *where* eseguendo la *query interna*

Interrogazioni annidate (*nested queries*)

```
# Errore!! La funzione di aggregazione non può essere calcolata sulla singola riga
SELECT s.sname
FROM sailors s
WHERE s.rating = MAX(s.rating );

#Query corretta
SELECT s.sname
FROM sailors s
WHERE s.rating = (SELECT MAX(s2.rating)
FROM sailors s2);

#Non corretta semanticamente. Perché la query qui sotto dà un risultato diverso?
SELECT s.sname
FROM sailors s
ORDER BY s.rating DESC
LIMIT 1;
```

Interrogazioni annidate (*nested queries*)

- Una query annidata può restituire:
  - Un singolo valore (con cui confrontare un valore di un attributo della query esterna)

```
select s.sname
from sailors s
where s.rating = (select max(s2.rating)
                  from sailors s2 );
```

  - Un insieme di valori tra i quali vogliamo verificare se vale una certa condizione (normalmente espressa con gli operatori IN, EXISTS, ANY o ALL)

```
select s.sname , s.rating
from sailors s
where s.sid in (select r.sid from reserves r);
```

Query indipendenti e correlate

- Una query annidata si dice:
  - **Indipendente:** quando può essere eseguita in modo autonomo, senza fare riferimento a nessuna colonna o valore della query esterna.
  - **Correlata:** quando la query interna fa riferimento a una o più colonne della query esterna. Questo significa che la subquery deve essere rieseguita per ogni riga elaborata dalla query esterna
- Differenze principali:
  - **Esecuzione***:* una subquery indipendente viene eseguita una sola volta, e il suo risultato viene utilizzato dalla query esterna, mentre una subquery correlata deve essere rieseguita per ogni riga elaborata dalla query esterna
  - **Indipendenza***:* le subquery indipendenti possono essere eseguite anche da sole, mentre le query correlate non hanno senso al di fuori della query esterna.

Esempi

- **Subquery indipendente**: trovare i nomi dei marinai che hanno un rating minore rispetto al rating medio

```
select s.sname
from sailors s
where s.rating < (select avg(s2.rating)
                  from sailors s2 );
```

- **Subquery correlata**: trovare id e rating dei marinai che hanno effettuato meno di 2 prenotazioni

```
select s.sid, s.rating
from sailors s
where 2 > (select count(*)
           from reserves r2
           where r2.sid = s.sid);
```

Casi d'uso

- Usare in un filtro valori calcolati dinamicamente
  - Esempio: marinai che hanno un rating minore rispetto al rating medio di tutti i marinai

```
select s.sname
from sailors s
where s.rating < (select avg(s2.rating)
                  from sailors s2 );
```



	sname
1	Brutus
2	Art
3	Bob

  - Perché non si poteva fare semplicemente così?

```
select s.sname
from sailors s
where s.rating < avg(s.rating );
```



Casi d'uso

- Eseguire confronti con risultati derivati
  - Esempio: marinaio / marinai che ha / hanno il rating più alto

```
select s.sname
from sailors s
where s.rating = (select max(s2.rating)
                  from sailors s2 );
```



	sname
1	Rusty
2	Zorba

- Applicare filtri che dipendano dalla verifica che certi valori esistano (o non esistano)
  - Esempio: trovare i marinai che non hanno mai fatto neanche una prenotazione

```
select s.sid
from sailors s
where not exists (select r.sid from reserves r
                  where r.sid = s.sid);
```



	sid
1	29
2	32
3	58
4	71
5	85
6	95

Casi d'uso

- Eseguire operazioni di aggregazione condizionale
  - Esempio: id e rating dei marinai che hanno effettuato meno di 2 prenotazioni

```
select s.sid , s.rating
from sailors s
where 2 > (select count(*)
           from reserves r2
           where r2.sid = s.sid);
```



	sid	rating
1	29	1
2	32	8
3	58	10
4	71	10
5	74	9
6	85	3
7	95	3

- Unire i risultati di una subquery con altre tabelle usando una JOIN
  - Esempi nel seguito

Interrogazioni annidate e operatori insiemistici di confronto

- Esempio: trovare i marinai che hanno un rating più alto di quello di uno qualsiasi tra i marinai che si chiamano Horatio

```
select *
from sailors s
where s.rating > any (select s2.rating
                      from sailors s2
                      where s2.sname = 'Horatio');
```



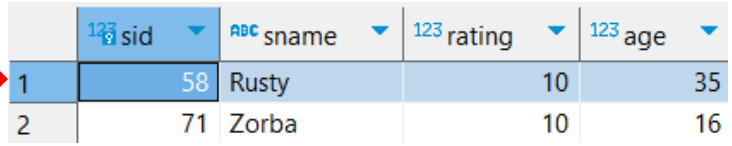
	sid	sname	rating	age
1	31	lubber	8	55,5
2	32	Andy	8	25,5
3	58	Rusty	10	35
4	71	Zorba	10	16
5	74	Horatio	9	35

  - ANY è vero se c'è un marinaio con rating più alto di un qualsiasi marinaio che si chiami Horatio (perché compare Horatio nei risultati?)
  - Se non ce ne sono, ANY restituisce FALSE e viene restituito l'insieme vuoto di tuple come risposta alla query

Interrogazioni annidate e operatori insiemistici di confronto

- Esempio 2: trovare i marinai che hanno un rating più alto di quello di tutti i marinai che si chiamano Horatio:

```
select *
from sailors s
where s.rating < all (select s2.rating
                      from sailors s2
                      where s2.sname = 'Horatio');
```



	sid	sname	rating	age
1	58	Rusty	10	35
2	71	Zorba	10	16

  - Qui la condizione deve valere rispetto a tutti i marinai che si chiamano Horatio
  - NB: se non c'è neanche un Horatio, ALL restituisce TRUE e viene restituito l'insieme di tutte le tuple di Sailor

Operatori insiemistici

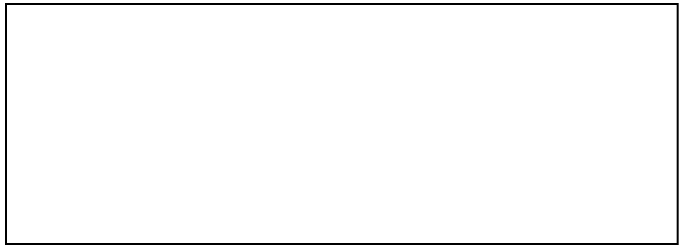
- Servono per effettuare operazioni insiemistiche tra due o più insiemi (compatibili) di tuple generate da altrettante query
- Essi sono:
  - **UNION** (**UNION ALL**): fa l'unione di due insiemi di tuple. UNION elimina gli eventuali duplicati, UNION ALL no
  - **INTERSECT**: restituisce solo le tuple comuni a due insiemi di tuple
  - **EXCEPT**: restituisce le tuple del primo insieme a patto che non compaiono nel secondo insieme (*set difference)*

Esempi di query con op. insiemistici

- Stabilire cosa restituiscono le seguenti query:

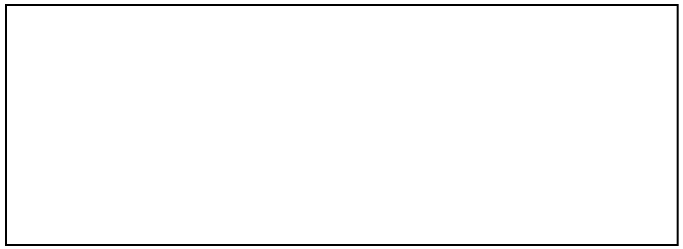
```
select s.sid from sailors s
except
select r.sid from reserves r ;
```





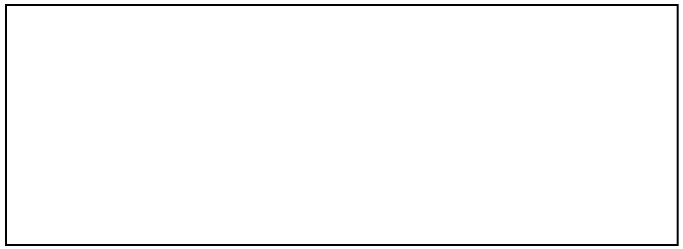
```
select s.sid from sailors s
intersect
select r.sid from reserves r ;
```





```
select r.sid
from reserves r JOIN boats b on r.bid = b.bid
where b.color = 'red'
union
select r.sid
from reserves r JOIN boats b on r.bid = b.bid
where b.color = 'blue';
```





Dove possono essere annidate le query

Le *sub query* possono essere annidati in diversi punti di un'interrogazione SQL:

- Nella clausola **FROM**: serve a costruire una tabella temporanea su cui eseguire il resto della query
- Nella clausola **WHERE**: serve a calcolare al volo un singolo valore su cui operare un confronto o un insieme di valori da confrontare con un valore già disponibile
- Nella clausola **HAVING** (vedi prossime slide): serve a calcolare un valore per filtrare i gruppi che devono essere restituiti (simile al WHERE, ma si opera su gruppi e non su righe)

SUBQUERIES, CTE, TEMP TABLES, VIEWS

Definizioni

- **Subquery**: una query annidata *(inner query)* all'interno di un'altra query (*outer query*)
- **Common Table Expression** (CTE): è un insieme di tuple temporaneo, costruito sulla base di una query. Esiste solo per il tempo di esecuzione della query principale
- **Tabella temporanea**: tabella che viene create e esiste temporaneamente per la durata di una sessione o di una transazione
- **Vista** (**view**): è una tabella virtuale creata mediante una query. Non memorizza i dati sul DB, ma è persistente (può essere richiamata in ogni momento all'interno di una query)

Esempio (*business logic*)

- Schema:
  - tabella '**sales**':

sales_id	employee_id	amount	sale_date

  - tabella '**employees**' :

employee_id	employee_name	department

- Trovare il totale delle vendite per ogni dipartimento nel mese corrente, eliminando i dipartimenti che hanno venduto per meno di $2000.

Esempio: uso di subquery

```
SELECT department, total_sales
FROM (
        SELECT e.department, SUM(s.amount) AS total_sales
        FROM sales s
        JOIN employees e ON s.employee_id = e.employee_id
        WHERE MONTH(s.sale_date) = MONTH(CURDATE())
        GROUP BY e.department
) AS SalesPerDepartment
WHERE total_sales > 2000;
```

Esempio: CTE

```
WITH DepartmentSales AS (
SELECT e.department, SUM(s.amount) AS total_sales
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE MONTH(s.sale_date) = MONTH(CURDATE())
GROUP BY e.department
)
SELECT department, total_sales
FROM DepartmentSales
WHERE total_sales < 2000;
```

Esempio: CTE – cont.

```
with GoodSailors as (
        select * from sailors s
        where s.rating > 8),
     BadSailors as (
        select * from sailors s2
        where s2.rating < 3)
select sid, sname, 'GOOD' from GoodSailors
union
select sid, sname, 'BAD' from BadSailors ;
```

Esempio: Tabella temporanea

```
CREATE TEMPORARY TABLE TempSales AS
SELECT e.department, SUM(s.amount) AS total_sales
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE MONTH(s.sale_date) = MONTH(CURDATE())
GROUP BY e.department;

SELECT department, total_sales
FROM TempSales
WHERE total_sales > 2000;
```

Esempio: Vista

```
CREATE VIEW DepartmentSalesView AS
SELECT e.department, SUM(s.amount) AS total_sales
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE MONTH(s.sale_date) = MONTH(CURDATE())
GROUP BY e.department;

SELECT department, total_sales
FROM DepartmentSalesView
WHERE total_sales < 2000;
```

Vantaggi e svantaggi delle tre soluzioni

Subquery:

- **Vantaggi:** le subquery possono essere annidate e utilizzate direttamente all'interno della struttura della query.
- **Limiti:** le subquery (in particolare quelle corrrelate) possono talvolta causare problemi di prestazioni se usate in modo eccessivo in query complesse, rendendole più difficili da ottimizzare e comprendere.

Vantaggi e svantaggi delle tre soluzioni

CTE:

- **Vantaggi:** la CTE migliora la leggibilità consentendo di calcolare separatamente la parte *DepartmentSales*.
- **Limiti:** il *result set* di una CTE è temporaneo e non può essere riutilizzato in più query

Vantaggi e svantaggi delle tre soluzioni

Tabelle temporanee:

- **Vantaggi:** la tabella temporanea può essere utilizzata più volte in diverse parti delle query, rendendola ideale per query complesse o quando è necessario riutilizzare il risultato in più passaggi/query.
- **Limitazione:** occupa spazio di archiviazione temporaneamente ed è specifica della sessione, il che significa che, una volta terminata la sessione / transazione, la tabella viene eliminata.

Vantaggi e svantaggi delle tre soluzioni

Viste:

- **Vantaggio:** le viste sono persistenti, il che significa che possono essere utilizzate ripetutamente senza dover riscrivere la logica ogni volta.
- **Limitazione:** le viste non sono adatte a operazioni complesse e su larga scala in cui i dati devono essere modificati frequentemente o interrogati in contesti diversi, poiché non memorizzano i dati ma interrogano ogni volta le tabelle sottostanti.

I JOIN TRA TABELLE

A cosa servono i JOIN

- Il JOIN è l'operazione che serve a unire informazioni che stanno in tabelle diverse
  - Per esempio, i dati anagrafici di un cliente con i dettagli del suo ordine
- Ci sono diversi tipi di JOIN, che corrispondono a diverse esigenze della business logic

JOIN(s)

- Consideriamo le 4 principali forme di JOIN:
  1. (INNER) JOIN: la condizione del JOIN è soddisfatta dalle tuple di entrambe le tabelle
  2. LEFT (OUTER) JOIN: restituisce tutte le tuple della tabella di sinistra e solo quelle che soddisfano la condizione del JOIN della relazione di destra
  3. LEFT (OUTER) JOIN: restituisce tutte le tuple della tabella di sinistra e solo quelle che soddisfano la condizione del JOIN della relazione di destra
  4. FULL (OUTER) JOIN*: tutte le tuple ogni volta che c'è un match in almeno una delle due tabelle



* MariaDB (come molti altri DBMS) **non** supporta il FULL OUTER JOIN!

INNER JOIN vs. PRODOTTO CARTESIANO

- Quando dobbiamo eseguire un INNER JOIN, abbiamo sempre due approcci alternativi:

1. Usare esplicitamente l'operatore JOIN … ON:

```
select s.sid , r.bid , r.`day`
from sailors s join reserves r on s.sid = r.sid
where r.bid = 103;
```

2. Mettere le tabelle interessate nella *relation-list* e specificare le condizioni utilizzando la clausola WHERE:

```
select s.sid , r.bid , r.`day`
from sailors s, reserves r
where s.sid = r.sid and r.bid = 103;
```

INNER JOIN vs. PRODOTTO CARTESIANO

- Le due query restituiscono esattamente lo stesso risultato:

```
select s.sid , r.bid , r.`day`
from sailors s join reserves r on s.sid = r.sid
where r.bid = 103;

select s.sid , r.bid , r.`day`
from sailors s, reserves r
where s.sid = r.sid and r.bid = 103;
```



	sid	bid	day
1	22	103	1998-08-10
2	31	103	1998-06-11
3	74	103	1998-08-09

- Anche se in teoria l'uso del comando JOIN … ON è raccomandato (soprattutto per chiarezza nel distinguere la condizione di JOIN dagli altri filtri), nella pratica è possibile utilizzare le due forme in modo interscambiabile
- Diverso è il caso in cui sia necessario usare forme di OUTER JOIN

LEFT / RIGHT OUTER JOIN

- Immaginiamo ora di voler conoscere quante prenotazioni ha fatto ogni marinaio
- Se usiamo l'*inner join* o il prodotto cartesiano, otteniamo questo risultato:

```
select s.sid , count(*)
from sailors s inner join reserves r
on s.sid = r.sid
group by s.sid ;
```

```
select s.sid , count(*)
from sailors s , reserves r
where s.sid = r.sid
group by s.sid ;
```



	sid	count(*)
1	22	4
2	31	3
3	64	2
4	74	1

LEFT / RIGHT OUTER JOIN

- Tuttavia, questo risultato non risponde alla nostra richiesta, perché non ci sono i marinai che non hanno mai prenotato una barca.
- Per farlo, dobbiamo usare un LEFT (OUTER) JOIN:

```
select s.sid , count(r.sid)
from sailors s left join reserves r
on s.sid = r.sid
group by s.sid ;
```



	sid	count(r.sid)
1	22	4
2	29	0
3	31	3
4	32	0
5	58	0
6	64	2
7	71	0
8	74	1
9	85	0
10	95	0

LEFT / RIGHT OUTER JOIN

- Immaginiamo ora di voler conoscere da quanti marinai è stata prenotata ogni barca
- In questo caso, usiamo un RIGHT (OUTER) JOIN:

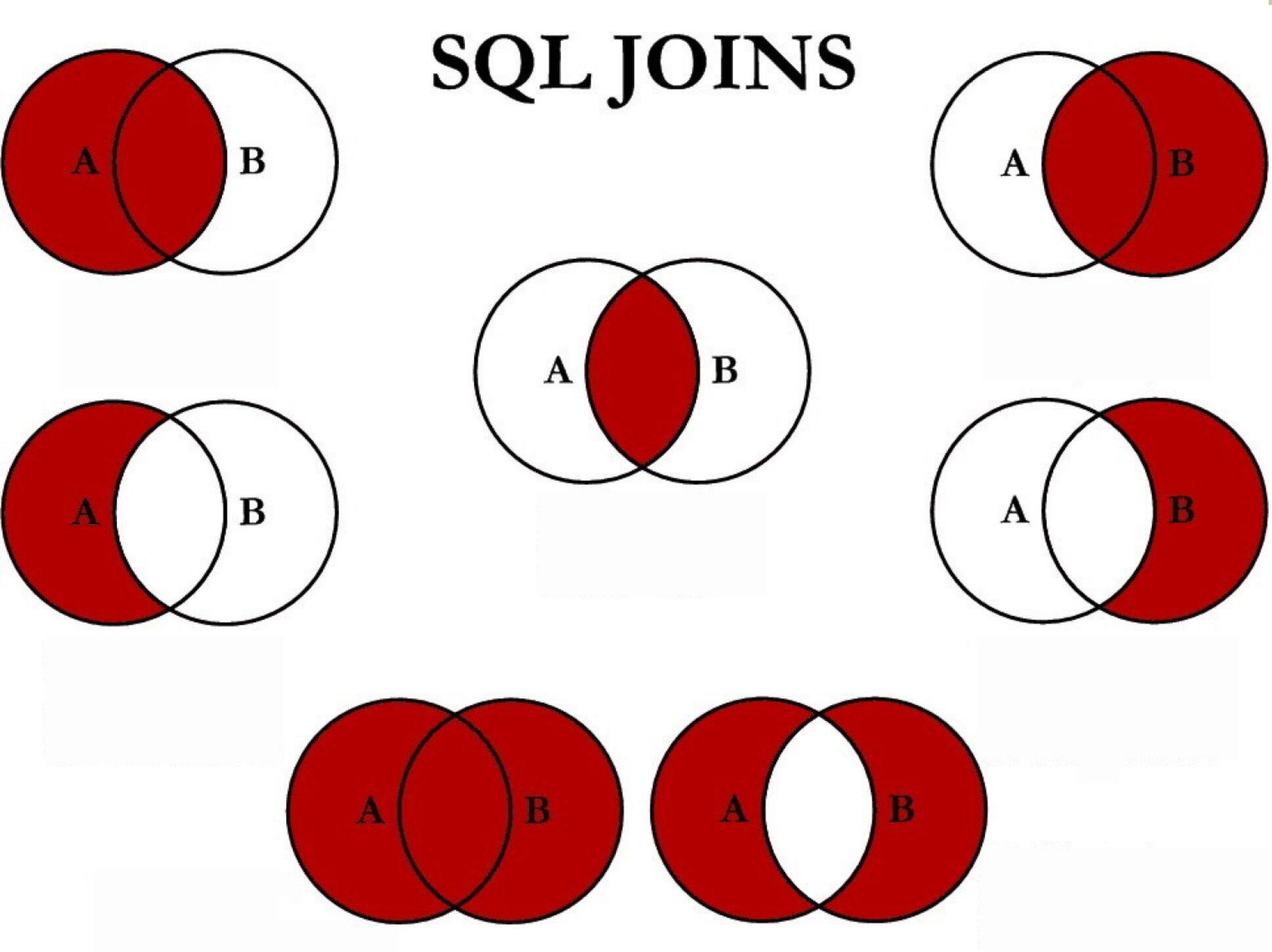
```
select b.bid, count(r.sid)
from sailors s
join reserves r on s.sid = r.sid
right join boats b on b.bid = r.bid
group by b.bid ;
```



	bid	count(r.sid)
1	101	2
2	102	3
3	103	3
4	104	2

- Con questa specifica istanza della BdD, sarebbe cambiato qualcosa se avessimo usato l'INNER JOIN al posto del RIGHT JOIN? Perché?

SQL JOIN(s) - II

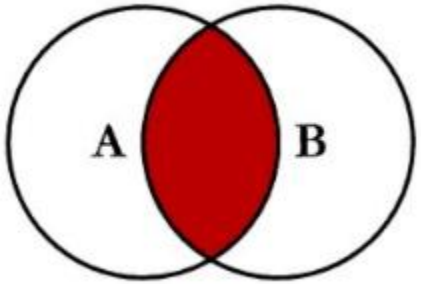


Cosa c'è di diverso rispetto ai casi precedenti?

SQL JOIN(s)

- I tre nuovi casi si differenziano da quelli visti finora perché NON includono le tuple che hanno una corrispondenza con una tupla dell'altra relazione
- Vengono talvolta chiamati LEFT / RIGHT /FULL ANTI JOIN
- Sono poco usati nello sviluppo di applicazioni, ma sono molto utili in progetti di *business intelligence* o *data science*
- Come possiamo ottenere gli ANTI JOIN dai JOIN che sappiamo usare?

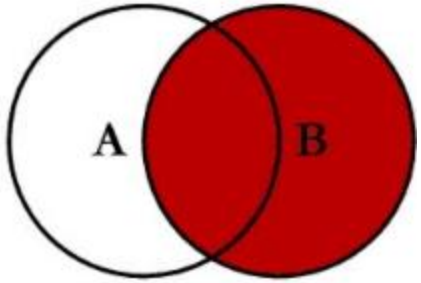
Tutti i marinai che hanno noleggiato una barca (INNER JOIN)




```
select *
from reserves r join sailors s
on r.sid = s.sid;
```

	sid	bid	day	sid	sname	rating	age
1	22	101	1998-10-10	22	Dustin	7	45
2	64	101	1998-05-09	64	Horatio	7	35
3	22	102	1998-10-10	22	Dustin	7	45
4	31	102	1998-10-11	31	lubber	8	55,5
5	64	102	1998-08-09	64	Horatio	7	35
6	22	103	1998-08-10	22	Dustin	7	45
7	31	103	1998-06-11	31	lubber	8	55,5
8	74	103	1998-08-09	74	Horatio	9	35
9	22	104	1998-07-10	22	Dustin	7	45
10	31	104	1998-01-11	31	lubber	8	55,5

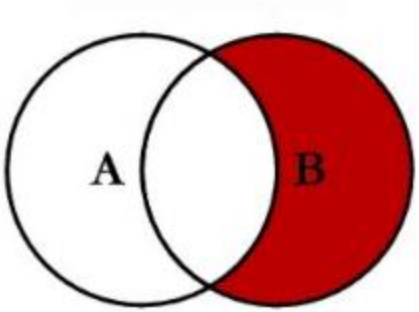
I noleggi effettuati da tutti i marinai noti (RIGHT JOIN)




```
select *
from reserves r right join sailors s
on r.sid = s.sid;
```

	sid	bid	day	sid	sname	rating	age
1	22	101	1998-10-10	22	Dustin	7	45
2	22	102	1998-10-10	22	Dustin	7	45
3	22	103	1998-08-10	22	Dustin	7	45
4	22	104	1998-07-10	22	Dustin	7	45
5	[NULL]	[NULL]	[NULL]	29	Brutus	1	33
6	31	102	1998-10-11	31	lubber	8	55,5
7	31	103	1998-06-11	31	lubber	8	55,5
8	31	104	1998-01-11	31	lubber	8	55,5
9	[NULL]	[NULL]	[NULL]	32	Andy	8	25,5
10	[NULL]	[NULL]	[NULL]	58	Rusty	10	35
11	64	101	1998-05-09	64	Horatio	7	35
12	64	102	1998-08-09	64	Horatio	7	35
13	[NULL]	[NULL]	[NULL]	71	Zorba	10	16
14	74	103	1998-08-09	74	Horatio	9	35
15	[NULL]	[NULL]	[NULL]	85	Art	3	22,5
16	[NULL]	[NULL]	[NULL]	95	Bob	3	63,5

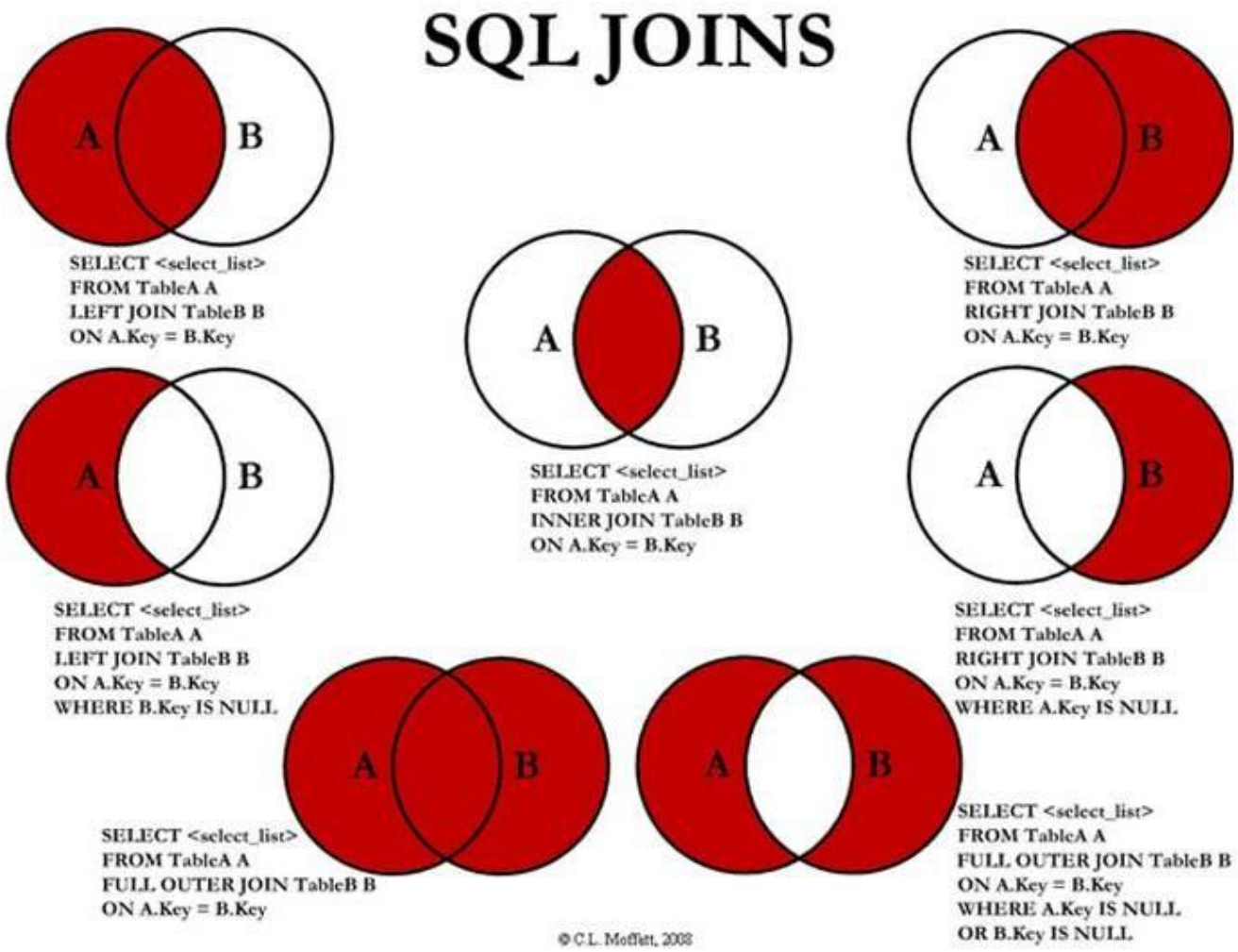
Tutti i marinai che non hanno noleggiato barche (RIGHT ANTI JOIN)




```
select *
from reserves r right join sailors s
on r.sid = s.sid
where r.sid is NULL;
```

	sid	bid	day	sid	sname	rating	age
1	[NULL]	[NULL]	[NULL]	29	Brutus	1	33
2	[NULL]	[NULL]	[NULL]	32	Andy	8	25,5
3	[NULL]	[NULL]	[NULL]	58	Rusty	10	35
4	[NULL]	[NULL]	[NULL]	71	Zorba	10	16
5	[NULL]	[NULL]	[NULL]	85	Art	3	22,5
6	[NULL]	[NULL]	[NULL]	95	Bob	3	63,5

JOIN(s) e ANTI JOIN(s) con RELATIVA QUERY



CENNI DI DDL PER SQL

Creazione di tabelle

Sintassi

```
CREATE TABLE table_name (
  column1 datatype constraint,
  column2 datatype constraint,
  column3 datatype constraint,
  ....
);
```

Esempio

```
CREATE TABLE Persons (
  PersonID int PRIMARY KEY,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255)
);
```

Vincoli (*contraints*)

- `NOT NULL` - Ensures that a column cannot have a NULL value
- `UNIQUE` - Ensures that all values in a column are unique
- `PRIMARY KEY` - Uniquely identifies each row in a table (a combination of a `NOT NULL` and `UNIQUE`)
- `FOREIGN KEY` - Establishes a link between data in two tables, and prevents action that will destroy the link between them
- `CHECK` - Ensures that the values in a column satisfies a specific condition
- `DEFAULT` - Sets a default value for a column if no value is specified
- `CREATE INDEX` - Creates indexes on columns to retrieve data from the database faster

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255) NOT NULL,
    Age int
);
```

```
CREATE TABLE Persons (
    ID int NOT NULL UNIQUE,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

```
CREATE TABLE Persons (
    ID int PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

```
CREATE TABLE Persons (
    ID int,
    LastName varchar(255),
    FirstName varchar(255),
    Age int,
    PRIMARY KEY (ID, LastName)
);
```

```
CREATE TABLE Persons (
    ID int,
    LastName varchar(255),
    FirstName varchar(255),
    Age int,
    CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)
);
```

Vincoli (*contraints*)

- `NOT NULL` - Ensures that a column cannot have a NULL value
- `UNIQUE` - Ensures that all values in a column are unique
- `PRIMARY KEY` - Uniquely identifies each row in a table (a combination of a `NOT NULL` and `UNIQUE`)
- `FOREIGN KEY` - Establishes a link between data in two tables, and prevents action that will destroy the link between them
- `CHECK` - Ensures that the values in a column satisfies a specific condition
- `DEFAULT` - Sets a default value for a column if no value is specified
- `CREATE INDEX` - Creates indexes on columns to retrieve data from the database faster

```sql
CREATE TABLE Orders (
    OrderID int PRIMARY KEY,
    OrderNumber int NOT NULL,
    PersonID int,
    CONSTRAINT fk_Person
    FOREIGN KEY (PersonID)
    REFERENCES Persons(PersonID)
);
```

```sql
CREATE TABLE Persons (
    ID int PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255) DEFAULT 'Sandnes'
);
```

```sql
CREATE TABLE Persons (
    ID int PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int CHECK (Age >= 18)
);
```

```sql
CREATE TABLE Persons (
    ID int PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255),
    CONSTRAINT chk_PersonAge CHECK (Age >= 18 AND City = 'Sandnes')
);
```

Popolare una tabella

Syntax 1

Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Syntax 2

If you insert values for ALL the columns of the table, you can omit the column names.

However, the order of the values must be in the same order as the columns in the table:

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

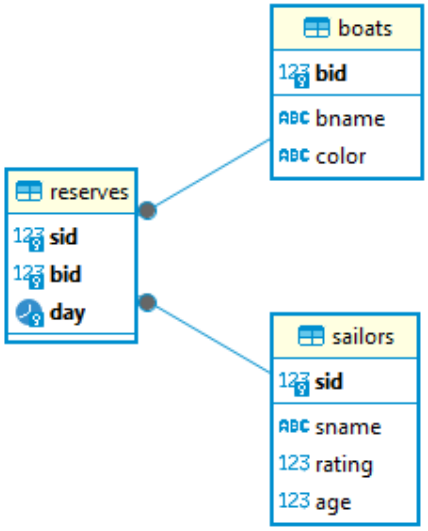
```
INSERT INTO Customers
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

LINEE GUIDA PRATICHE PER PROGETTARE QUERY SQL

Studia lo schema

R1: accertati di aver compreso bene lo schema del database che devi interrogare!

- Se disponibile, consulta il diagramma con lo schema relazionale in forma grafica
- Memorizza i nomi delle tabelle e dei loro attributi (per quanto possibile) o trascrivili su un pezzo di carta
- Verifica il dominio di ogni attributo (per capire quali operazioni potrai applicare a ognuno di essi)
- Accertati di aver compreso bene i vincoli di integrità referenziale (ti serviranno per i JOIN)
- Se di aiuto, prova a trascrivere in linguaggio naturale che cosa rappresenta ogni tabella



Leggi bene la richiesta!

R2: Leggi con molta attenzione la formulazione della query in linguaggio naturale e cosa chiede esattamente

- Elenca gli attributi che sono richiesti nell'output (→ *target-list*)
- Associa ognuno di questi attributi alla tabella dove compaiono (→ costruisci la *relation-list*)
- Identifica i filtri che dovranno essere applicati per eliminare le tuple indesiderate
- Chiediti se possono esserci duplicati e quindi se sia necessario utilizzare il DISTINCT sui risultati

Cerca le aggregazioni!

R3: Prova a identificare parole o espressioni che suggeriscono l'uso di funzioni di aggregazione:

- Restituire il valore medio … → AVG?
- Trovare il totale … → SUM?
- Il numero di clienti che … → COUNT?
- Il fatturato annuale diviso per venditore … → SUM + GROUP BY?
- I venditori il cui fatturato annuale è superiore a 20.000 Euro … → SUM + GROUP BY + HAVING?

Comprendi i vincoli di IR

R4: Identifica con precisione le condizioni che dovrai usere per fare il JOIN di due o più tabelle

- Identifica gli attributi che dovrai utilizzare (tipicamente chiave primaria → chiave esterna)
- Ricorda che il WHERE è una funzione che si applica tupla per tupla (e non su più tuple contemporaneamente)
- Rifletti con attenzione sul tipo di JOIN che dovrai applicare per esser semanticamente fedele alla richiesta (INNER, OUTER LEFT, OUTER RIGHT, ANTI JOIN, …)

Cerca le query annidate!

R5: Identifica se la query rientra in uno dei casi d'uso per la costruzione di query annidate

- Devo fare un confronto tra il valore di un attributo in una tupla e un valore calcolato dinamicamente da una sub query
- Devo verificare se il valore di un attributo in una tupla appartiene a un insieme di valori → IN
- Devo confrontare un valore con un insieme di valori generati da una sub query → ANY, ALL
- Devo verificare se il valore di in attributo in una sub query correlata produce almeno una tupla come risultato → EXISTS

Trova eventuali operazioni insiemistiche

R5: Ove è il caso, usa diagrammi (tipo Venn) per determinare le operazioni insiemistiche di cui hai bisogno nella query

- Verifica se devi unire tuple con lo stesso schema ma che provengono da tabelle diverse (→ UNION)
- Verifica se devi trovare tuple comuni tra insiemi di tuple che provengono da diverse tabelle con lo stesso schema (→ INTERSECT)
- Verifica se devi trovare tuple che compaiono in un insieme ma non in un altro (→ EXCEPT)
- …

Verifica la coerenza (semantica) dei risultati

R6: Verifica che intuitivamente il risultato della query sia quello che corrisponde alla richiesta

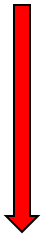
- Scomponi la query in query più semplici per verificare che ogni «pezzo» della query restituisca quello che ti aspettavi
- Verifica che non ci siano valori che violano qualche filtro (es. se chiedi i marinai con rating > 5, non devono comparire marinai che nella tabella Sailors hanno valore 4 ☺)
- Verifica che il numero di tuple restituite sia intuitivamente della cardinalità corretta
- Controlla di aver restituito tutti e soli gli attributi che erano richiesti nella query
- Verifica di non aver dimenticato di riordinare le tuple di output (se richiesto)
- Verifica se hai usato il DISTINCT dove è necessario

QUERY AGGIUNTIVE

Esempi di query di base

Q0: sname ed età dei marinai

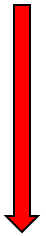
```
select s.sname, s.age
from sailors s;
```



	sname	age
1	Dustin	45
2	Brutus	33
3	lubber	55,5
4	Andy	25,5
5	Rusty	35
6	Horatio	35
7	Zorba	16
8	Horatio	35
9	Art	22,5
10	Bob	63,5

Q0.1 sname ed età dei marinai che hanno più di 50 anni

```
select s.sname, s.age
from sailors s
where s.age > 50;
```

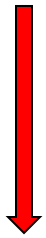


	sname	age
1	lubber	55,5
2	Bob	63,5

Esempi di query di base

(Q1) Trovare i nomi dei marinai che hanno prenotato la barca #103

```
SELECT  S.sname
FROM    Sailors S, Reserves R
WHERE   S.sid=R.sid AND R.bid=103
```

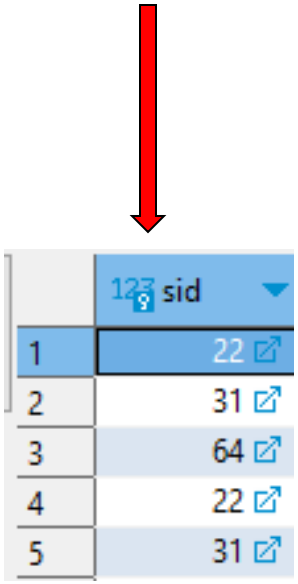


	ABC sname
1	Dustin
2	lubber
3	Horatio

Esempi

sid di marinai che hanno noleggiato una barca rossa

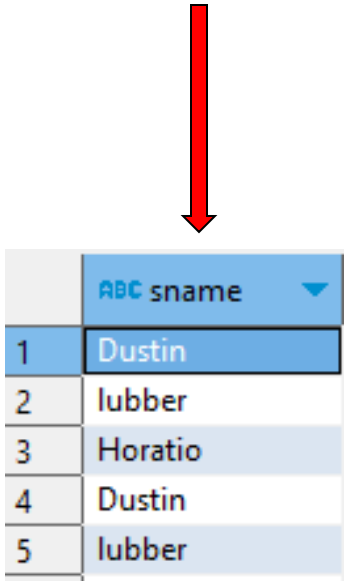
```
SELECT   R.sid
FROM     Boats B, Reserves R
WHERE    B.bid = R.bid AND B.color = 'red'
```



	sid
1	22
2	31
3	64
4	22
5	31

sname di marinai che hanno noleggiato una barca rossa

```
SELECT   S.sname
FROM     Sailors S, Reserves R, Boats B
WHERE    S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
```

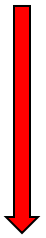


	sname
1	Dustin
2	lubber
3	Horatio
4	Dustin
5	lubber

Esempi

Uso di espressioni aritmetiche e pattern nelle query

```
SELECT  S.age, age1=S.age-5, 2*S.age AS age2
FROM  Sailors S
WHERE  S.sname LIKE 'B_%B'
```

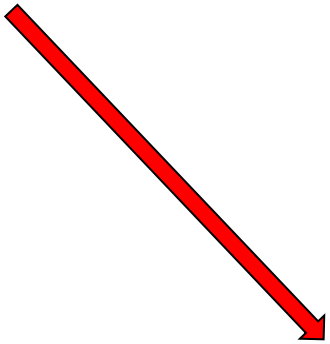


	123 age	123 age1	123 age2
1	63,5	58,5	127

Esempi

(Q6) Nomi di marinai che hanno prenotato sia una barca rossa, sia una barca verde

```
SELECT DISTINCT s.sid
from Sailors s , Boats b1 , Reserves r1 , Boats b2, Reserves r2
where s.sid = r1.sid and r1.bid = b1.bid and
s.sid = r2.sid and r2.bid = b2.bid
and b1.color = 'red' and b2.color = 'green';
```



```
SELECT s.sid from Sailors s , Boats b , Reserves r
where s.sid = r.sid and r.bid = b.bid and b.color = 'red'
INTERSECT
SELECT s1.sid from Sailors s1 , Boats b1 , Reserves r1
where s1.sid = r1.sid and r1.bid = b1.bid and b1.color ='green';
```



	sid
1	22
2	31

Esempi

(Q19) Trovare i sid dei marinai che hanno prenotato barche rosse ma non barche verdi

```
SELECT r.sid from Boats b , Reserves r
where r.bid = b.bid and b.color = 'red'
EXCEPT
SELECT r.sid from Boats b , Reserves r
where r.bid = b.bid and b.color ='green';
```

